



SOVEREIGN OS DIM

Guide Développeur

Version 36.0 - 8 sections - référence technique

GHT Psy Sud Paris - Fondation Vallée + Paul Guiraud

Généré le 22 / 07 / 2026

Adam Beloucif - adam.beloucif@psysudparis.fr

Ce guide est destiné aux développeurs, aux équipes DSI et aux contributeurs du projet. Il documente l'architecture, la stack, le bridge HTTP, le module ML, la sécurité et la CI/CD. Pour le guide d'utilisation quotidienne (TIM, médecin DIM, chef de pôle), voir Sovereign_OS_DIM_Guide.pdf.

Dépôt - https://github.com/Adam-Blf/sovereign_os_dim - Stack - Python 3.12 + C# .NET 8 + WebView2 + XGBoost

Sommaire

1. **Architecture générale**
2. **Stack technique et dépendances**
3. **Bridge HTTP et intégration PHP**
4. **Module ML - configuration et entraînement**
5. **Sécurité et conformité RGPD**
6. **CI/CD - pipeline et qualité de code**
7. **Performances et benchmarks**
8. **Contribuer au projet**

Architecture générale

Vue d'ensemble

Sovereign OS DIM est un monolithe desktop PyInstaller. Le runtime desktop C# (.NET 8 + WebView2) héberge une fenêtre native Windows dans laquelle tourne un frontend HTML/CSS/JS. Le backend Python est lancé en sous-processus et exposé via un bridge HTTP local (127.0.0.1 uniquement).

Couches

Couche	Technologie	Rôle
Desktop shell	C# .NET 8 + WebView2	Fenêtre native, IPC, packaging
Frontend	HTML + Tailwind + Chart.js	UI rendue dans WebView2
Backend API	Python 3.12 + FastAPI/aiohttp	Traitement PMSI, bridge HTTP
Données	SQLite (Microsoft.Data.Sqlite)	MPI persisté, résolutions IDV
ML	XGBoost, LightGBM, scikit-learn	Assistance TIM (3 modèles)
PDF	fpdf2 >= 2.8 (LGPL)	Génération guides + exports

Flux de données

1. L'utilisateur interagit avec le frontend (WebView2).
2. Les requêtes passent par le bridge HTTP (127.0.0.1:8765) avec un token Bearer (env var SOVEREIGN_BRIDGE_TOKEN).
3. Le backend Python traite les fichiers ATIH en parallèle (ThreadPoolExecutor, max 8 workers) et écrit en SQLite.
4. Les résultats sont renvoyés en JSON au frontend.

Le bridge HTTP ne doit jamais écouter sur 0.0.0.0 - 127.0.0.1 uniquement. Toute modification expose les données patient.

Stack technique et dépendances

Backend Python

Package	Version	Usage
Python	3.12	Runtime principal
fpdf2	>= 2.8	Génération PDF (LGPL)
xgboost	>= 2.0	format_detector + collision_ri
lightgbm	>= 4.0	Benchmark ML (challenger)
scikit-learn	>= 1.4	ddn_validity (RandomForest) +
pandas	>= 2.0	Chargement dataset synthétique
pypdf	>= 4.0	Enrichissement bookmarks PDF
PyInstaller	>= 6.0	Packaging .exe portable

Frontend

HTML/CSS vanille + Tailwind CSS (CDN embarqué). Chart.js pour les graphiques. Aucun framework JS (React/Vue) - volontairement léger pour rester embarquable dans WebView2 sans build step. ES modules natifs (import/export).

C# Desktop Shell

Microsoft.Web.WebView2 pour l'affichage HTML. Microsoft.Data.Sqlite pour l'accès direct au MPI depuis C#. WinForms comme conteneur de fenêtre. Le shell C# lance le backend Python via Process.Start() et détecte son port via stdout.

Installation de l'environnement de développement

```
# Python (backend + ML + PDF)
pip install -r requirements.txt

# Tests
pytest tests/ -v

# Lint + sécurité
ruff check .
bandit -r backend/
mypy backend/

# Build .exe
python build.py
```

Bridge HTTP et intégration PHP

Démarrage du bridge

Le bridge HTTP est un serveur FastAPI/aiohttp écoutant exclusivement sur 127.0.0.1. Il est protégé par un token Bearer passé en variable d'environnement. Le port par défaut est 8765.

```
# Démarrage minimal
SOVEREIGN_BRIDGE_TOKEN=secret python bridge.py --port 8765

# Côté PHP (client)
export SOVEREIGN_BRIDGE_URL=http://127.0.0.1:8765
export SOVEREIGN_BRIDGE_TOKEN=secret
php -S 127.0.0.1:8080 -t php
```

Endpoints disponibles

Méthode	Route	Auth	Description
GET	/health	Non	Ping - retourne {status: ok}
GET	/mpi/stats	Oui	KPI Dashboard (fichiers, IPP,
POST	/mpi/process	Oui	Lance traitement d'un dossier
GET	/mpi/collisions	Oui	Liste des collisions IDV
POST	/mpi/resolve	Oui	Résout une collision (IPP + DD
GET	/export/csv	Oui	Export MPI CSV normalisé
GET	/export/txt/{id}	Oui	Export .txt sanitized d'un fic

Authentification

Toutes les routes sauf /health nécessitent un header Authorization: Bearer <TOKEN>. Le token est comparé à SOVEREIGN_BRIDGE_TOKEN via hmac.compare_digest() pour éviter les timing attacks. Retourne HTTP 401 si absent ou invalide.

CORS

Origine autorisée - null (WebView2) et http://127.0.0.1 uniquement. Toute autre origine est bloquée par les headers CORS restrictifs. Méthodes autorisées - GET, POST, OPTIONS.

Exemple PHP

```
<?php
$url = getenv('SOVEREIGN_BRIDGE_URL');
$token = getenv('SOVEREIGN_BRIDGE_TOKEN');
$ch = curl_init("$url/mpi/stats");
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    "Authorization: Bearer $token",
    "Accept: application/json",
]);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

```
$resp = json_decode(curl_exec($ch), true);  
echo $resp['ipp_unique']; // ex. 4821
```

Module ML - configuration et entraînement

Modèles embarqués

Modèle	Algorithme	Tâche	Métrique clé
format_detector.json	XGBoost tuned	Classification format ATIH (58	accuracy 0,77 - F1 0,70
collision_risk.json	XGBoost tuned	Scoring risque collision IDV	AUC 1,00 - F1 1,00
ddn_validity.pkl	RandomForest	Détection DDN suspecte	AUC 0,86 - acc 0,99

Entraînement

Les modèles sont entraînés sur un dataset 100 % synthétique fidèle aux spécifications ATIH 2000-2026 (58 variantes). Aucun fichier patient réel n'est utilisé (RGPD art. 9).

```
# Entraînement standard (50k samples)
python -m backend.ml.train

# Entraînement plus robuste (200k samples)
python -m backend.ml.train --samples 200000 --seed 42

# Réutiliser un parquet existant (gain ~10s)
python -m backend.ml.train --data-cache data/cache.parquet

# Résultat : backend/ml/models/*.json + *.pkl
#           + training_metadata.json (leaderboard + hashes SHA-256)
```

Pipeline de benchmark

Pour chaque tâche, 4 algorithmes sont évalués en cross-validation stratifiée (5-fold) - XGBoost default, XGBoost tuned (Optuna, 30 trials), LightGBM, RandomForest. Le winner est sélectionné par AUC (binaire) ou F1 macro (multiclasse) et sérialisé. Le leaderboard complet est disponible dans training_metadata.json.

Ajouter un nouveau format ATIH

```
# 1. Editer le catalogue des specs
# backend/ml/synthetic.py :: ATIH_SPECS

ATIH_SPECS.append({
    'format': 'RPS',
    'version': 'P15',
    'year_from': 2027,
    'line_length': 166,
    'ipp_start': 20,
    'ipp_len': 20,
    'ddn_start': 40,
})
```

```
# 2. Relancer l'entraînement
```

```
python -m backend.ml.train --samples 200000
```

Audit des modèles

training_metadata.json contient pour chaque modèle - date d'entraînement, nombre de samples, seed, leaderboard complet des 4 candidats avec métriques test, hash SHA-256 du fichier modèle sérialisé. Ce fichier est versionné en Git pour tracabilité réglementaire.

Sécurité et conformité RGPD

Périmètre réseau

Le bridge HTTP écoute exclusivement sur 127.0.0.1. Toute tentative de bind sur 0.0.0.0 est bloquée par validation explicite au démarrage. Aucun port n'est ouvert vers l'extérieur du poste.

Authentification bridge

Token Bearer via SOVEREIGN_BRIDGE_TOKEN (variable d'environnement uniquement, jamais hardcodé). Comparaison via hmac.compare_digest() (constant time). HTTP 401 retourné sans détail sur l'erreur.

Validation des chemins

os.path.abspath() + vérification que le chemin résolu reste sous le dossier autorisé (path traversal guard). Côté C# - SafePath.validate() applique la même règle. Les liens symboliques pointant hors du dossier cible sont rejetés.

Données patient (RGPD art. 9)

Les IPP et DDN sont traités uniquement en mémoire RAM pendant le traitement de lot. Après traitement, seule la base SQLite locale contient les données (chiffrée au repos via SQLCipher, clé dérivée du token). Aucune télémétrie, aucun envoi réseau, 100 % local.

Audit log art. 30 RGPD

Toute résolution IDV (qui, quand, IPP, DDN retenue, DDN rejetées) est journalisée dans audit.log (append-only). Le log est lisible par le DPO et l'ARS sur demande. Rétention 5 ans minimum (instruction DGOS/PF2/2020/143).

Anonymisation exports recherche

Option k-anonymity $k \geq 5$ disponible pour les exports destinés à la recherche (SNDS, Health Data Hub). Hash SHA-256 tronqué (12 chars) appliqué aux IPP dans les exports pseudonymisés. Conforme MR-007.

Secrets et CI

```
# Variables d'environnement requises (jamais committées)
SOVEREIGN_BRIDGE_TOKEN=<token>    # bridge auth
SQLCIPHER_KEY=<clé>                # chiffrement SQLite

# CI scan sécurité (GitHub Actions)
bandit -r backend/ -ll              # vulnérabilités Python
ruff check .                        # linting
mypy backend/                       # typage statique
```

CI/CD - pipeline et qualité de code

Pipeline GitHub Actions

Déclenchement - push sur main ou PR vers main. Environnement - ubuntu-latest, Python 3.12.

Étape	Commande	Bloquant
Lint	ruff check .	Oui
Typage	mypy backend/	Oui
Sécurité	bandit -r backend/ -ll	Oui
Tests	pytest tests/ -v --tb=short	Oui
Build	python build.py --check	Non (avertissement)

Conventions de commit

Format - <type>(<scope>): <description>

Types - feat, fix, docs, refactor, test, chore.

Scope - backend, frontend, ml, bridge, guide, ci.

Langue - anglais impératif, minuscules.

Tirets longs (-) interdits dans les messages de commit.

Exemples valides -

feat(ml): add RPS P15 format to ATIH_SPECS

fix(bridge): prevent path traversal in /export/txt

Pull Requests

Branche de base - main. Branche de travail - <type>/<description-courte>. Tests pytest verts obligatoires avant merge. Review par au moins 1 membre du projet. Squash merge - un seul commit par PR dans main.

Release

Cycle trimestriel - V36 juillet 2026, V37 octobre 2026. Tag Git : vXX.Y.Z (semver). GitHub Release auto-générée avec changelog depuis les commits. Binaire PyInstaller attaché à la release (Sovereign_OS_DIM.exe).

Régénérer les guides PDF

```
# Guide métier (TIM, médecin DIM, chef de pôle)
```

```
python tools/generate_guide.py
```

```
# Guide développeur (DSI, contributeurs)
```

```
python tools/generate_guide_dev.py
```

```
# Mode d'emploi court (utilisateurs finaux)
```

```
python tools/generate_manual.py
```

Performances et benchmarks

Conditions de mesure

Poste de référence - Intel Core i5 8e gen, 16 Go RAM, SSD NVMe. Système : Windows 11 Pro. Python 3.12, xgboost 2.0, sqlite 3.45.

Backend - traitement PMSI

Opération	Mesure	Notes
Scan 1 000 fichiers (12 dossier)	1,2 s	Heuristique regex
Identification 1 fichier	180 µs	Regex + longueur ligne
Traitement lot 10 fichiers / 1	5 s	8 workers en pic
Ecriture MPI SQLite 150k ligne	3 s	Bottleneck principal
Chargement MPI 500k lignes	< 400 ms	Index sur ipp

Module ML

Opération	Mesure	Notes
Chargement 3 modèles au démarr	180 ms	XGBoost JSON + RandomForest pk
format_detector (1 ligne)	< 1 ms	Inférence XGBoost
collision_risk (1 IPP)	< 1 ms	Inférence XGBoost
ddn_validity (1 ligne)	< 1 ms	Inférence RandomForest
Batch 10 000 lignes (tous modè)	~150 ms	Sans parallélisme

Frontend (WebView2)

Opération	Mesure	Notes
Chargement initial dashboard	< 400 ms	MPI 50k IPP
Rafraîchissement post-traiteme	< 200 ms	Recalcul agrégats en mémoire
Rendu donut Chart.js	< 80 ms	Canvas HTML5
Export PDF organigramme 3p	1,2 s	fpdf2 vectoriel

Goulot principal - écriture SQLite pour les très gros lots (> 1 M lignes). Solution : traiter par trimestres successifs ou passer à WAL mode (journal_mode=WAL).

Contribuer au projet

Dépôt

https://github.com/Adam-Blf/sovereign_os_dim

Licence - MIT (code) + LGPL (fpdf2).

Issues GitHub pour bugs et suggestions.

Contact direct - adam.beloucif@psysudparis.fr

Structure du dépôt

Dossier / Fichier	Contenu
backend/	Python : traitement PMSI, brid
backend/ml/	Module ML : train.py, syntheti
frontend/	HTML + Tailwind + Chart.js (to
php/	Client PHP du bridge HTTP
tools/	Générateurs PDF (guide, guide
tests/	Tests pytest
docs/	Screenshots, PDFs générés
bridge.py	Entrée du bridge HTTP (main Py
main.py	Entrée de l'application deskto
build.py	Script PyInstaller (packaging

Ajouter un format ATIH

1. Editer backend/ml/synthetic.py - ajouter l'entrée dans ATIH_SPECS (format, version, year_from, line_length, positions IPP/DDN).
2. Relancer l'entraînement ML (python -m backend.ml.train --samples 200000).
3. Mettre à jour le détecteur heuristique dans backend/data_processor.py (regex + longueur).
4. Ajouter un test dans tests/ pour le nouveau format.
5. Documenter dans docs/research/pmsi_formats_history.md.

Tests

```
# Lancer tous les tests
pytest tests/ -v

# Tests d'un module spécifique
pytest tests/test_bridge_security.py -v

# Avec couverture
pytest tests/ --cov=backend --cov-report=html
```

Merci de contribuer ! Les contributions améliorant la couverture des formats ATIH ou la robustesse du ML sont particulièrement bienvenues.

Références et ressources

Documentation officielle ATIH

Notice technique 2026 -
<https://www.atih.sante.fr/notice-technique-nouveautes-pmsi-mco-had-smr-psychiatrie-2026-0>
Formats PMSI 2026 (Excel) - <https://www.atih.sante.fr/formats-pmsi-2026-0>
Catalogue formats historiques - [format-pmsi.fr](https://www.atih.sante.fr/format-pmsi.fr)
Plateforme DRUIDES - <https://www.epmsi.atih.sante.fr>

RGPD et conformité

RGPD UE 2016/679 - art. 9 (données de santé), art. 30 (registre de traitement).
HAS - Instruction DGOS/PF2 no 2019-116 (identitovigilance).
ANS - Référentiel national d'identito-vigilance 2023.
Health Data Hub - MR-007 (pseudonymisation SNDS).
Instruction DGOS/PF2/2020/143 - rétention 5 ans.

Dépôt et support

GitHub - https://github.com/Adam-Blf/sovereign_os_dim
Issues - https://github.com/Adam-Blf/sovereign_os_dim/issues
Contact - adam.beloucif@psysudparis.fr
Alternance EFREI M1 Data Engineering - Fondation Vallée 2025-2027.

Ce guide développeur est généré automatiquement depuis `tools/generate_guide_dev.py` - mettre à jour le contenu dans ce fichier pour l'enrichir.